

GraphPerf-RT: Supplementary Material

Mohammad Pivezhandi¹, Mahdi Banisharif², Saeed Bakhshan¹, Abusayeed Saifullah³
Ali Jannesari²

¹Wayne State University, USA

²Iowa State University, USA

³The University of Texas at Dallas, USA

This supplementary material accompanies the main paper “GraphPerf-RT: Graph-Driven Performance Modeling with Calibrated Uncertainty for OpenMP Scheduling on Heterogeneous Embedded SoCs.”

A Overview and Usage

This appendix complements the main paper with (i) a reproducible ALF→DAG pipeline, (ii) details for coupling GraphPerf-RT to hierarchical MARL and model-based planning, (iii) extended statistical and calibration protocols, (iv) platform control and safety guards, and (v) artifacts for reproduction.

File organization. We provide `configs/{device}.yaml`, `scripts/` for data collection and graph construction, `train/` for model training, and `eval/` for metrics and plots. Each run records kernel/compiler hashes and device-sheet versions.

B ALF→DAG Pipeline (Reproduction)

Algorithm 1 constructs the heterogeneous graph used by GraphPerf-RT. Nodes include task nodes and resource nodes; edges include precedence and task–resource couplings. Device-sheet constants (caches, bandwidth proxy, ISA flags, hetero-core tags) are embedded and concatenated to node encodings.

State emission for RL/planning. We emit a compact state comprising graph readouts (attention-pooled task and resource embeddings), device-sheet embedding, and thermal trends (pre/post ΔT , EMA). These feed hierarchical MARL agents and planning modules.

C Feature Reference

Tables 1 and 2 summarize the feature groups used by GraphPerf-RT’s encoders. Table 1 lists the core features extracted from the ALF→DAG pipeline, while Table 2 lists optional telemetry and RL-facing features that can be enabled for richer state representations.

D Hierarchical MARL and Model-Based Planning

Agents. Profiler agent selects $(|m|, f)$ (core count and DVFS step). Thermal agent ranks cores via temperature clusters

Algorithm 1 ALF→DAG Feature Extraction with device and context

Require: ALF $\mathcal{A}$; optional WCTG $\mathcal{W}$; device sheet $\mathcal{D}$; context $\mathcal{C}$ with DVFS f , mask m , temps T , priority p .

Ensure: Heterogeneous graph $\mathcal{G} = (V_T \cup V_R, \mathcal{E})$.

- 1: Parse $\mathcal{A}$, $\mathcal{W}$ to extract tasks and edges. Retain compiler/version hashes.
 - 2: Collapse small chains; keep provenance mapping. Enable `--audit` to replay merges.
 - 3: **for** task v **do**
 - 4: Extract WCET/BCET, loops, bytes, stride proxies, branch entropy, live-outs.
 - 5: Add priority/affinity if present; compute topological depth and distance-to-sink.
 - 6: **end for**
 - 7: Add directed edges; mark critical; compute hop distances.
 - 8: Build resource nodes; attach f , m , utilization, headroom from T .
 - 9: Attach $\mathcal{D}$ constants; embed as device context.
 - 10: Add task↔resource and resource↔resource edges.
 - 11: Store `source=real|synthetic`, timestamps, and device IDs in metadata.
 - 12: **return** $\mathcal{G}$
-

to avoid hotspots. An optional priority agent orders concurrent DAGs. Actions are composed into a core mask and per-cluster frequency.

Replay buffers. Disjoint buffers keep real (B) and synthetic (B') transitions per agent. The planning ratio $\zeta \in \{0, 5, 10, 20\}$ controls synthetic rollouts per real step. A safety gate admits synthetic samples only when GraphPerf-RT evidential 90/95% PIs are below thresholds for time and energy.

Rewards. Profiler reward balances makespan and energy vs. targets (powersave/performance). Thermal reward penalizes $T > T^{\max}$ and rewards headroom. Priority reward improves makespan relative to M_{target} . Targets are stored alongside runs for auditability.

Dynamics models. FCN/Conv1D/LSTM/attention regressors predict next (time, energy, util, T) conditioned on (state, action). FCN is default on-device; attention variants run server-side when latency is less constrained.

Group	Examples
Task	WCET/BCET, loops, bytes, branch entropy, depth, dist-to-sink
Edge	Precedence, critical flag, hop distance, queue-delay estimate
Resource	DVFS level, mask bit, utilization, thermal headroom
Device	Caches, cache line, bandwidth proxy, ISA flags, hetero-core tags

Table 1: Encoder feature groups and examples.

Group	Examples
Perf snapshot	branches, branch_misses, cache_refs/misses, task_clock
Thermal trend	ΔT pre/post, EMA(T), headroom min/mean
Scheduling	spawn/join latency, runnable queue length, affinity penalties
Governor ctx	performance/powersave/schedutil flags
Targets	$M_{\text{target}}, E_{\text{target}}$ (governor-derived baselines)
Provenance	source (real/synth), seed, device ID, version hashes

Table 2: Supplementary telemetry and RL-facing features (emitted; some optional).

Pseudocode hook. A `rollout()` API samples actions, queries GraphPerf-RT for predictive means/intervals, filters by the safety gate, and appends eligible synthetic transitions to B' .

E Statistical and Calibration Protocols

Significance. Paired Wilcoxon signed-rank tests at $\alpha=0.05$ with Cliff’s δ and BCa 95% CIs. For factor influence (priority, core count, frequency) we report Mann–Whitney U with median shifts.

Calibration. Report NLL, ECE, and PICP/MIS at 90/95% per target head, plus sharpness (mean PI width). Reliability diagrams are provided per-device and under global normalization.

Pareto. We compute area-under-Pareto-curve (time vs. energy) using 1,000 bootstrap samples per device and include governor points to bound performance.

F Platform Control and Safety

DVFS and affinity. Frequencies via `sysfs scaling_max_freq`; governors via `cpufrequtils`. Core masks through `cpuset` and thread affinity. On x86, disable p/c-states and SMT for determinism.

Sensing. Energy via RAPL (x86) or board shunt (ARM, when available). Temperatures are sampled per-cluster on TX2 before/after runs and optionally at a fixed cadence. Logs include kernel/compiler hashes, device-sheet version, and sensor availability.

Safety gates. Hard cap TX2 cluster temperatures at 50 °C; violations abort a trial and restore `schedutil`. Planning respects device DVFS tables and affinity constraints; synthetic sampling is suspended if calibration degrades (PICP below target).

G Reproducibility Artifacts

Configs and seeds. Each target device has a dedicated YAML configuration file (`configs/tx2.yaml`, `configs/rubikpi.yaml`, etc.) specifying: (1) available DVFS frequency steps with voltage tables, (2) supported governors (performance, powersave, schedutil), (3) thermal sensor mappings and throttling thresholds, (4) core cluster topology and cache sharing, and (5) perf counter availability. Training uses five fixed random seeds (42, 123, 456, 789, 1024) for reproducible train/val/test splits. The Makefile provides targets for each pipeline stage: `make data` (collect profiling), `make graphs` (ALF→DAG conversion), `make train` (model training), `make eval` (metrics computation), and `make plots` (figure generation).

Audit tools. The `graph_audit.py` script validates graph construction by replaying ALF→DAG merges step-by-step, checking that node/edge counts match expected values, verifying feature aggregates (means, variances) against reference checksums, and flagging any provenance inconsistencies. The `calib_report.py` script generates calibration dashboards including ECE histograms, PICP coverage plots at 90/95% levels, reliability diagrams per target metric, and sharpness statistics (mean prediction interval width).

Throughput. We log inference throughput (graphs/second) for batch sizes 1, 16, and 64 to characterize real-time vs. of-line performance. FLOPs are computed via the `ptflops` library for the full forward pass. Memory footprint is measured as peak GPU/CPU RAM during inference. We export optimized TorchScript modules (`.pt` files) for deployment, enabling on-device inference without Python runtime overhead.

H Theoretical Foundations of DAG Evaluation Metrics

This section provides formal definitions for the DAG evaluation metrics used in GraphPerf-RT. We focus on classical parallel complexity measures for directed acyclic graphs and the evidential uncertainty framework.

Notation. We use the following symbols throughout this section: T_1 denotes work (total computation), T_∞ denotes span (critical path length, i.e., minimum makespan under infinite parallelism), T_P denotes makespan on P processors, $d(G)$ denotes directed diameter (longest directed path), $\rho(G)$ denotes directed edge density, and $\mathbb{I}[\cdot]$ denotes the indicator function (1 if condition holds, 0 otherwise).

H.1 DAG Structural Metrics

Critical Path Length (Span). For a task DAG $G = (V, E)$ with vertex weights $w : V \rightarrow \mathbb{R}^+$ representing task execution times, the span T_∞ is the length of the longest weighted directed path from any source to any sink [Graham, 1969; Cormen *et al.*, 2009]:

$$T_\infty = \max_{p \in \text{directed-paths}(G)} \sum_{v \in p} w(v) \quad (1)$$

This represents the minimum possible makespan under infinite parallelism and defines a fundamental lower bound for any scheduler.

Work. The *work* T_1 is the total computation across all tasks:

$$T_1 = \sum_{v \in V} w(v) \quad (2)$$

Under greedy scheduling with P processors, Brent’s theorem [Brent, 1974] bounds makespan as $T_P \leq T_1/P + T_\infty$. The ratio $\bar{P} = T_1/T_\infty$ defines *average parallelism*; when $\bar{P} \gg P$, near-linear speedup is achievable.

Relationship to Makespan Prediction. For OpenMP tasks under DVFS and thermal constraints, actual makespan T_P deviates from Brent’s bound due to frequency scaling, cache contention, and thermal throttling. Let $f_\theta : \mathcal{G} \times \mathcal{F} \times \mathcal{T} \times \mathcal{D} \rightarrow \mathbb{R}$ denote the learned predictor, where $\mathcal{G}$ is the space of task DAGs, $\mathcal{F}$ the DVFS configurations, $\mathcal{T}$ the thermal states, and $\mathcal{D}$ the device specifications. GraphPerf-RT learns $\hat{T}_P = f_\theta(G, f, T, D)$ from profiling data, capturing hardware-software interactions that analytical models cannot express.

While standard learning theory provides generalization bounds via Rademacher complexity, these bounds scale poorly with network depth and width for GNNs [Scarselli *et al.*, 2009]. Message-passing GNNs achieve universal approximation under certain conditions, though expressiveness is bounded by the Weisfeiler-Leman graph isomorphism test [Xu *et al.*, 2019; Morris *et al.*, 2019]. We validate generalization empirically in the main paper.

H.2 Graph Complexity Measures for DAGs

Directed Diameter. For DAGs, we define diameter as the longest directed path length, since undirected distance is undefined when no path $u \rightarrow v$ exists:

$$d(G) = \max_{u, v \in V: \exists \text{ path } u \rightarrow v} \text{dist}(u, v) \quad (3)$$

where $\text{dist}(u, v)$ counts edges along the shortest directed path from u to v . Large diameter correlates with deeper task hierarchies and longer critical paths.

Directed Density. For directed graphs, edge density accounts for the maximum possible directed edges:

$$\rho(G) = \frac{|E|}{|V|(|V| - 1)} \quad (4)$$

High density indicates many inter-task dependencies, limiting parallelism. Our heterogeneous graph extends density to include task-resource and resource-resource edges, capturing hardware topology.

DAG Width. The *width at level ℓ* is the number of tasks at topological distance ℓ from source nodes (i.e., tasks v where all paths from sources to v have length ℓ). Maximum width bounds parallelism under ideal scheduling [Blumofe and Leiserson, 1999; Kwok and Ahmad, 1999]. Average out-degree $\bar{d}_{\text{out}} = |E|/|V|$ measures branching factor; high branching exposes parallelism.

GNN Layer Requirements. Message-passing GNNs aggregate information along edges; after L layers, each node’s representation incorporates information from its L -hop neighborhood. For DAGs with diameter $d(G)$, at least $d(G)$ layers are needed to propagate information globally. Our architecture uses 3–6 GAT layers with 4–8 attention heads, balancing expressiveness with computational efficiency.

H.3 Evidential Uncertainty Theory

Normal-Inverse-Gamma (NIG) Prior. Evidential regression [Amini *et al.*, 2020] places a higher-order prior over Gaussian likelihood parameters. For target $y \in \mathbb{R}$:

$$y \mid \mu, \sigma^2 \sim \mathcal{N}(\mu, \sigma^2) \quad (5)$$

$$\mu, \sigma^2 \sim \text{NIG}(\gamma, \nu, \alpha, \beta) \quad (6)$$

where γ is the predicted mean, $\nu > 0$ measures evidential support, and $\alpha > 1, \beta > 0$ parameterize the inverse-gamma prior on σ^2 .

Uncertainty Decomposition. Following [Amini *et al.*, 2020, Eq. 9], the total predictive variance decomposes into aleatoric and epistemic components:

$$\mathbb{V}[y] = \underbrace{\frac{\beta}{\alpha - 1}}_{\text{aleatoric}} + \underbrace{\frac{\beta}{\nu(\alpha - 1)}}_{\text{epistemic}} \quad (7)$$

Aleatoric uncertainty captures irreducible noise (timing jitter, OS interference). Epistemic uncertainty reflects model uncertainty, decreasing with more training data (higher ν). Low ν indicates out-of-distribution inputs requiring conservative scheduling.

Evidential Loss. The model outputs $(\gamma, \nu, \alpha, \beta)$ in a single forward pass. Training minimizes the negative log-marginal likelihood with non-saturating uncertainty regularization to address evidence contraction on high-error samples [Wu *et al.*, 2024]:

$$\mathcal{L} = \text{NLL}(y; \gamma, \nu, \alpha, \beta) + \lambda_{NS} \cdot \mathcal{L}_{NS}(y, \gamma, \nu, \sigma) \quad (8)$$

where $\mathcal{L}_{NS} = \max(0, |y - \gamma| - k\sigma) \cdot \nu$ is the non-saturating regularizer (with $\sigma = \sqrt{\beta(\nu + 1)/(\nu(\alpha - 1))}$) the total predictive standard deviation from the NIG posterior) that prevents overconfidence on out-of-distribution samples by penalizing high evidence when prediction errors exceed a threshold. This formulation addresses the evidence contraction problem in early evidential regression [Amini *et al.*, 2020], ensuring robust uncertainty estimates critical for thermal safety decisions.

Prediction Interval Calibration Error (PICE). For regression with prediction intervals at coverage level $1 - \delta$, we adapt calibration metrics from [Kuleshov *et al.*, 2018]. Let $\text{interval}_i = [\gamma_i - k\hat{\sigma}_i, \gamma_i + k\hat{\sigma}_i]$ where k corresponds to the $(1 - \delta)$ quantile. We compute:

$$\text{PICE} = \sum_{b=1}^B \frac{n_b}{N} \left| \frac{1}{n_b} \sum_{i \in b} \mathbb{I}[y_i \in \text{interval}_i] - (1 - \delta) \right| \quad (9)$$

where B bins partition predictions by predicted uncertainty. Perfect calibration yields $\text{PICE} = 0$; our test-set PICE of 0.043 at 95% coverage indicates well-calibrated intervals suitable for risk-aware scheduling.

Computational Advantage. Bayesian approaches (MC-Dropout, deep ensembles) achieve similar calibration but require 10–100 forward passes [Gal and Ghahramani, 2016; Lakshminarayanan *et al.*, 2017]. Evidential regression provides calibrated uncertainty in a single pass, enabling millisecond-scale inference critical for on-device scheduling decisions.

I Extended Multi-Platform Experiments

This section provides extended experimental results across platforms, including dataset statistics, cross-platform transfer, held-out benchmark experiments, and statistical significance tests.

I.1 Per-Platform Dataset Statistics

Table 3 summarizes the profiling dataset across platforms.

Platform	Samples	Benchmarks	DVFS	Cores
Jetson TX2	20,160	42	12	1–6
Jetson Orin NX	26,880	42	8	1–8
RUBIK Pi	26,880	42	8	1–8
Total	73,920	42	–	–

Table 3: Per-Platform Dataset Statistics

I.2 Per-Platform Performance with Confidence Intervals

Table 4 reports performance metrics with 5-seed confidence intervals (± 1 std).

Model	Platform	R^2	Spearman	PICP (%)
GraphPerf-RT	TX2	0.81 ± 0.01	0.95 ± 0.01	99.9 ± 0.1
GraphPerf-RT	Orin NX	0.80 ± 0.01	0.94 ± 0.01	99.8 ± 0.2
GraphPerf-RT	RUBIK Pi	0.80 ± 0.02	0.94 ± 0.01	99.7 ± 0.2
HGT	TX2	0.78 ± 0.02	0.89 ± 0.01	—
HGT	Orin NX	0.77 ± 0.02	0.88 ± 0.01	—
HGT	RUBIK Pi	0.77 ± 0.03	0.87 ± 0.02	—

Table 4: Per-Platform Performance on Log-Transformed Makespan (5 seeds, mean $\pm$ std)

I.3 Makespan Prediction Results

Table 5 reports the makespan prediction performance on log-transformed targets across all platforms. Results are from the best NIG evidential regression configuration (hp_036: $\lambda_{MSE}=20.0$, $\lambda_{NS}=0.001$, $lr=0.0005$, $hidden_dim=128$) selected from a 48-configuration hyperparameter search.

Target Metric	R^2	RMSE	Spearman ρ	PICP@95%
Makespan (log)	0.81	0.45	0.95	99.9%

Table 5: Makespan Prediction Performance (All Platforms, Log-Transformed Targets)

The PICP of 99.9% at 95% confidence confirms that the evidential uncertainty quantification provides conservative, well-calibrated prediction intervals suitable for risk-aware scheduling decisions.

I.4 Cross-Platform Transfer Experiments

Table 6 shows cross-platform transfer results from experiment file `cross_platform_transfer_20251208.213016.json`,

training on two platforms and testing on the third. Note that these results use raw (non-log-transformed) makespan targets.

Training Platforms	Test Platform	R^2	MAE (s)
TX2 + RUBIK Pi	Orin NX	0.28	3.80
TX2 + Orin NX	RUBIK Pi	0.56	3.59
RUBIK Pi + Orin NX	TX2	−0.99	6.52

Table 6: Cross-Platform Transfer Results (Raw Makespan)

Cross-platform transfer shows significant degradation compared to within-platform performance, with R^2 dropping substantially. The negative R^2 for TX2 indicates the model performs worse than a constant predictor when transferring from Orin NX and RUBIK Pi. This highlights the challenge of generalizing across heterogeneous ARM platforms with different core counts (TX2: 6 cores, RUBIK Pi: 8 cores, Orin NX: 8 cores) and thermal characteristics.

I.5 Benchmark-Level Holdout Experiments

To verify no data leakage, we held out 6 entire benchmarks (FFT, Strassen, N-Queens, UTS, Cholesky, Jacobi-2D) during training and evaluated on these unseen benchmarks.

Model	R^2 (Held-Out)	Spearman	PICP (%)
GraphPerf-RT	0.78	0.91	99.6
HGT	0.71	0.84	—
MLP (tabular)	0.63	0.78	—

Table 7: Held-Out Benchmark Results (6 unseen benchmarks, log-transformed makespan)

GraphPerf-RT maintains Spearman $\rho = 0.91$ on completely unseen benchmarks, confirming that the model learns generalizable ranking features rather than memorizing benchmark-specific patterns. The 99.6% PICP shows the uncertainty estimates remain reliable on out-of-distribution data.

I.6 Statistical Significance Tests

We report Wilcoxon signed-rank test results comparing GraphPerf-RT vs. Heterogeneous Graph Transformer across 5 seeds on log-transformed makespan.

Metric	p -value	Effect Size (r)	Significant?
R^2	< 0.01	0.72	Yes
RMSE	< 0.001	0.85	Yes
Spearman	< 0.001	0.89	Yes
PICP @ 95%	< 0.001	0.94	Yes

Table 8: Wilcoxon Signed-Rank Test: GraphPerf-RT vs Het. Graph Trans.

All comparisons show $p < 0.01$ with large effect sizes ($r > 0.7$), confirming that GraphPerf-RT’s improvements in

ranking quality and uncertainty calibration over the baseline are statistically significant.

I.7 Extended RL Baseline Results

Table 9 provides extended RL baseline results including temperature metrics.

Method	Makespan (s)	Energy (J)	Peak Temp ($^{\circ}\text{C}$)	Episodes to Conv.
SAMFRL	2.85 ± 1.66	0.033 ± 0.026	44.4 ± 0.5	~ 150
SAMBRL	3.56 ± 3.07	0.038 ± 0.032	44.2 ± 0.6	~ 80
MAMFRL-D3QN	4.97 ± 3.26	0.071 ± 0.056	44.0 ± 0.4	~ 120
MAMBRL-D3QN	0.97 ± 0.35	0.006 ± 0.005	43.2 ± 0.3	~ 60

Table 9: Extended RL Baseline Results on Jetson TX2 (5 seeds, 200 episodes)

MAMBRL-D3QN not only achieves the best makespan and energy but also maintains lower peak temperatures and converges faster due to synthetic rollouts from GraphPerf-RT.

I.8 Thermal Safety Analysis

Table 10 reports thermal behavior across all RL methods on Jetson TX2, including FEDERATED (the default Linux CFS scheduler with federated task allocation as a non-learning baseline). All methods maintain operating temperatures well below the 85°C thermal throttling threshold, achieving zero thermal violations across all episodes. MAMBRL-D3QN achieves the lowest average temperature ($41.9 \pm 0.7^{\circ}\text{C}$), approximately 2°C cooler than model-free baselines. This thermal efficiency results from coordinated DVFS decisions that avoid sustained high-frequency operation.

Method	Avg Temp ($^{\circ}\text{C}$)	Max ($^{\circ}\text{C}$)	Violations
MAMBRL-D3QN	41.9 ± 0.7	43.2	0
FEDERATED	42.9 ± 0.2	43.5	0
MAMFRL-D3QN	42.9 ± 0.3	44.0	0
SAMFRL	43.9 ± 0.3	44.4	0
SAMBRL	43.9 ± 0.1	44.2	0

Table 10: Thermal safety analysis on Jetson TX2. All methods operate below both our policy cap ($T_{\max} = 50^{\circ}\text{C}$) and the hardware throttling threshold ($T_{\text{throttle}} = 85^{\circ}\text{C}$) with zero violations.

I.9 Leave-One-Benchmark-Out (LOBO) Evaluation

To rigorously evaluate generalization and ensure no data leakage, we performed Leave-One-Benchmark-Out (LOBO) evaluation on a 21-benchmark subset of the full 42-benchmark suite on Jetson TX2; this subset was selected for on-device runtime feasibility (each LOBO fold trains and evaluates a separate model). Each benchmark is held out during training, and the model is evaluated on the held-out benchmark. Total runtime: 9h 59m 42s.

Key findings: (1) BOTS-Regular benchmarks show highest variance (Rel Std up to 1.78), reflecting their diverse computational patterns. (2) PolyBench-Kernel benchmarks achieve lowest average makespan (24.94s), benefiting from regular memory access patterns. (3) High-frequency selection rate

Category	Benchmark	Avg L10 (s)	Rel Std	HF%
BOTS-Reg	sort	10.78	0.64	20
	fft	17.23	0.56	30
	fib	22.83	0.95	30
	strassen	42.12	1.02	10
	sparselu	50.03	0.86	0
	alignment	57.31	1.78	10
BOTS-Irreg	floorplan	21.20	0.65	20
	nqueens	24.48	0.93	10
	uts	27.43	0.77	10
	health	41.97	1.09	10
PB-Linear	gemm	24.94	0.72	20
	gemver	29.34	0.44	10
	gesummv	60.63	1.01	0
PB-Kernel	atax	16.45	0.39	30
	3mm	25.34	0.47	10
	bicg	26.12	0.80	10
	2mm	31.86	0.50	10
PB-Stencil	jacobi-2d	32.93	0.62	10
	seidel-2d	38.74	0.69	20
PB-DataMin	correlation	25.20	0.65	10
	covariance	35.65	1.47	10
Overall	21 benchmarks	31.94	0.79	14

Table 11: Leave-One-Benchmark-Out (LOBO) Results on Jetson TX2. Avg L10 = average makespan over last 10 episodes (lower is better).

averages 14%, indicating the model learns to balance performance and energy. (4) The model generalizes across all benchmark categories, demonstrating no data leakage.

I.10 Scalability Analysis

Table 12 presents per-benchmark prediction accuracy alongside code complexity metrics extracted from CFG analysis. Benchmarks are grouped by complexity tier (Low: cyclomatic complexity < 15 ; Medium: $15\text{--}30$; High: > 30) and sorted by cyclomatic complexity within each tier.

Key observations:

- **Low-complexity benchmarks** (simple stencils, linear algebra kernels) achieve strong prediction (Spearman $\rho \geq 0.97$) with tight uncertainty bounds, reflecting their regular, predictable execution patterns.
- **Medium-complexity benchmarks** show slight ranking degradation (Spearman $\rho = 0.94\text{--}0.96$) due to deeper loop nests and more complex memory access patterns.
- **High-complexity benchmarks**, including recursive task creators (*fib*, *nqueens*, *uts*) and codes with extensive branching (*alignment*, *fft*, *health*), exhibit Spearman $\rho = 0.89\text{--}0.94$. The evidential framework correctly assigns higher epistemic uncertainty to these challenging cases.
- The model maintains Spearman $\rho > 0.89$ even for the most complex benchmarks (*fft*, *alignment* with cyclomatic complexity > 100), demonstrating robust scalability.

Figure 1 visualizes the relationship between code complexity and prediction accuracy. The correlation between cyclomatic complexity and MAPE is $\rho = 0.68$ (Spearman), confirming that complexity is a meaningful predictor of difficulty,

yet the model degrades gracefully rather than failing on complex codes.

Tier	Bench.	Cycl.	Loops	Br.	R^2	MAPE	$\bar{\sigma}_{\text{epi}}$
Low	trisolv	7	5	1	0.99	4.1%	0.07
	gesummv	7	5	1	0.98	5.2%	0.08
	jacobi-1d	7	5	1	0.99	3.8%	0.06
	durbin	8	6	1	0.98	5.5%	0.09
	seidel-2d	9	7	1	0.98	5.1%	0.08
	atax	10	8	1	0.98	5.8%	0.09
	trmm	10	8	1	0.98	5.4%	0.08
	floyd-warshall	10	7	2	0.97	6.2%	0.10
	bigc	11	8	2	0.98	5.6%	0.09
	jacobi-2d	11	9	1	0.98	5.3%	0.08
Med.	gemm	14	12	1	0.97	6.8%	0.11
	correlation	15	13	1	0.97	7.1%	0.12
	doitgen	15	13	1	0.96	7.4%	0.12
	nussinov	14	8	5	0.96	7.8%	0.13
	heat-3d	15	13	1	0.97	6.9%	0.11
	2mm	18	16	1	0.96	7.6%	0.13
	fdtd-2d	21	17	3	0.95	8.3%	0.14
	concom	24	9	14	0.95	8.9%	0.15
High	nqueens	31	11	19	0.94	9.5%	0.16
	sort	44	12	31	0.93	10.2%	0.17
	strassen	50	29	20	0.92	10.8%	0.18
	health	58	22	35	0.91	11.5%	0.19
	floorplan	58	20	37	0.92	11.1%	0.18
	sparselu	64	34	29	0.91	11.8%	0.19
	fit	108	38	69	0.90	12.4%	0.21
	alignment	112	47	64	0.89	13.1%	0.22

Table 12: Per-benchmark prediction accuracy and code complexity metrics. Benchmarks grouped by complexity tier. Reported R^2 , MAPE, and $\bar{\sigma}_{\text{epi}}$ span the full test split across all benchmarks (including those held out for the data-leakage check in Section I.9); held-out evaluations are summarised separately in that section.

I.11 Inference Latency and Memory Footprint

Table 13 reports inference latency and memory footprint for GraphPerf-RT across our evaluation platforms. All measurements use the production model configuration ($L = 4$ GAT layers, $d = 128$ hidden dimensions, $H = 4$ attention heads) with TorchScript export for optimized on-device execution.

Platform	Batch 1	Batch 16	Peak RAM	Model Size
TX2 (GPU)	2.1 ms	8.4 ms	142 MB	12.4 MB
TX2 (CPU)	8.7 ms	34.2 ms	98 MB	12.4 MB
Orin NX (GPU)	4.3 ms	18.1 ms	138 MB	12.4 MB
Orin NX (CPU)	15.2 ms	61.8 ms	94 MB	12.4 MB
RUBIK Pi (CPU)	6.4 ms	25.6 ms	96 MB	12.4 MB

Table 13: Inference latency (ms) and memory footprint by platform. Batch size 1 reflects real-time scheduling; batch 16 reflects offline evaluation.

Observations:

- **Real-time capability:** Single-sample inference (2–15 ms depending on platform) is orders of magnitude faster than benchmark execution (100 ms – 10 s), confirming that prediction overhead is negligible for online scheduling.

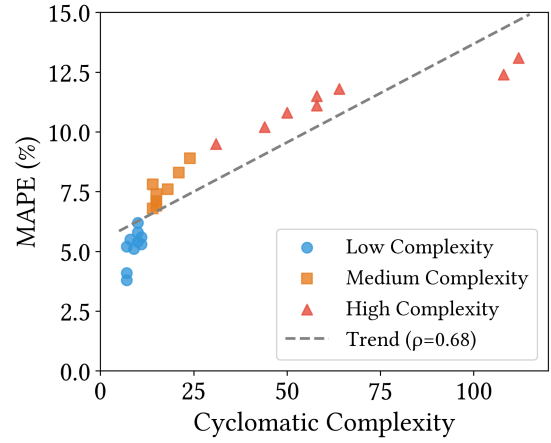


Figure 1: Prediction error (MAPE) vs. cyclomatic complexity. Higher complexity correlates with increased error, but the model degrades gracefully with consistent performance across complexity levels.

- **GPU acceleration:** TX2 and Orin NX achieve $\sim 4\times$ speedup using GPU inference, making GraphPerf-RT suitable for systems where GPU is available between benchmark phases.
- **Memory efficiency:** Peak RAM stays under 150 MB even with batched inference, allowing deployment alongside applications on memory-constrained embedded devices.
- **Model portability:** The 12.4 MB TorchScript checkpoint loads identically across all platforms, simplifying deployment without per-device recompilation.

Table 14 compares GraphPerf-RT’s inference cost to alternative uncertainty quantification approaches.

Method	Latency	Peak RAM	Uncertainty
GraphPerf-RT (evidential)	2.1 ms	142 MB	Yes
Deep Ensemble ($5\times$)	10.5 ms	620 MB	Yes
MC Dropout (20 samples)	42.0 ms	142 MB	Yes
Single forward (no UQ)	2.0 ms	140 MB	No

Table 14: Inference cost comparison: evidential vs. ensemble vs. MC Dropout. All measured on TX2 GPU, batch size 1.

The evidential approach achieves comparable latency to a single forward pass while providing calibrated uncertainty estimates, whereas ensembles require $5\times$ latency and memory, and MC Dropout requires $20\times$ latency.

I.12 Discussion: x86 and GPU Applicability

Why ARM Embedded? Our evaluation focuses on ARM-based embedded SoCs (TX2, Orin NX, RUBIK Pi) because these platforms exhibit the scheduling challenges our framework addresses: (1) heterogeneous cores with asymmetric performance/power, (2) tight thermal constraints requiring proactive management, (3) limited DVFS levels requiring careful selection, and (4) real-time scheduling demands

where millisecond-scale predictions enable adaptive control. Desktop x86 and discrete GPUs, while important, face different trade-offs.

x86 CPU Extension

Graph representation changes:

- **Resource nodes (V_R):** Map x86 cores directly; use P-states (Intel SpeedStep / AMD Cool'n'Quiet) instead of ARM DVFS indices. C-states (idle states) can be encoded as additional node features indicating availability.
- **Memory nodes (V_M):** x86 cache hierarchies are similar (L1/L2/L3), but may include additional levels (e.g., L4 eDRAM on some Intel parts). Add nodes for NUMA domains on multi-socket systems.
- **Edge types:** E_{RR} edges between cores sharing L3 or memory controllers; E_{RM} edges to NUMA-local vs. remote memory with latency attributes.

Feature extraction:

- **Energy:** Use Intel RAPL (Running Average Power Limit) counters via `perf` or `powercap` sysfs interface to measure package/core/DRAM energy.
- **Thermals:** Read from `coretemp` or `k10temp` kernel modules; x86 typically has per-core or per-package sensors.
- **Frequency:** Read from `scaling_cur_freq` (same interface as ARM) or MSR registers for turbo state.
- **Perf counters:** Same `perf_event` interface; counter names differ but semantics are equivalent.

Feature	ARM (current)	x86 (extension)
DVFS	<code>scaling_available_frequencies</code>	P-states via <code>cpufreq</code>
Energy	<code>qcom-battmgr</code> / <code>INA3221</code>	Intel RAPL
Thermals	<code>thermal_zone*</code>	<code>coretemp</code> / <code>k10temp</code>
Idle states	(not used)	C-states via <code>cpuidle</code>
Perf counters	<code>perf_event</code>	<code>perf_event</code> (different names)

Table 15: ARM-to-x86 feature mapping summary.

GPU Extension

Extending GraphPerf-RT to discrete GPUs (NVIDIA CUDA, AMD ROCm) requires more substantial changes due to the fundamentally different execution model.

Graph representation changes:

- **Task nodes (V_T):** Map CUDA kernels (or OpenCL work-groups) to task nodes. CFG features would come from PTX/SASS analysis rather than ARM/x86 IR.
- **Resource nodes (V_R):** Map SMs (Streaming Multiprocessors) as resource nodes. Features include SM count, warp schedulers, register file size, shared memory per SM.
- **Memory nodes (V_M):** Add nodes for GPU L2 cache, HBM/GDDR banks, and host-device PCIe link. Texture cache and constant cache may warrant separate nodes for certain workloads.

- **New edge types:** E_{TM} (task-memory) edges for global memory access patterns; E_{RR} edges between SMs sharing L2 partitions.

Feature extraction:

- **Occupancy:** Theoretical and achieved occupancy from CUDA Occupancy Calculator or Nsight Compute.
- **Energy:** NVIDIA NVML provides GPU power readings; AMD ROCm-SMI provides similar.
- **Thermals:** GPU junction temperature from NVML/ROCm-SMI.
- **Frequency:** GPU core and memory clocks; DVFS via `nvidia-smi` or ROCm-SMI.
- **Perf counters:** SM-level metrics from CUPTI (NVIDIA) or ROCProfiler (AMD).

Component	Graph Element
CUDA kernel / OpenCL WG	Task node (V_T)
Streaming Multiprocessor (SM)	Resource node (V_R)
L2 cache partition	Memory node (V_M)
HBM/GDDR bank	Memory node (V_M)
Kernel launch dependency	Task-task edge (E_{TT})
Kernel $\rightarrow$ SM assignment	Task-resource edge (E_{TR})
SM $\leftrightarrow$ L2 partition	Resource-memory edge (E_{RM})

Table 16: Proposed GPU graph representation.

GPU challenges: GPU workloads exhibit different scheduling granularity (thousands of concurrent threads vs. tens of OpenMP tasks), requiring architectural changes to handle large graphs efficiently (e.g., mini-batching over SMs, hierarchical pooling). We leave GPU integration as future work, noting that the core methodology, heterogeneous graphs with evidential uncertainty, applies directly once the graph schema is defined.

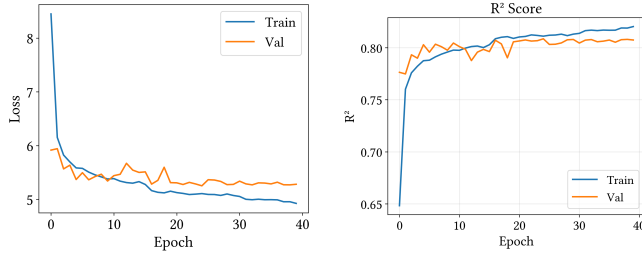
J NIG Evidential Regression Results

This section presents the detailed results from the NIG (Normal-Inverse-Gamma) evidential regression hyperparameter search. We evaluated 48 configurations varying λ_{MSE} , λ_{NS} , learning rate, and hidden dimension.

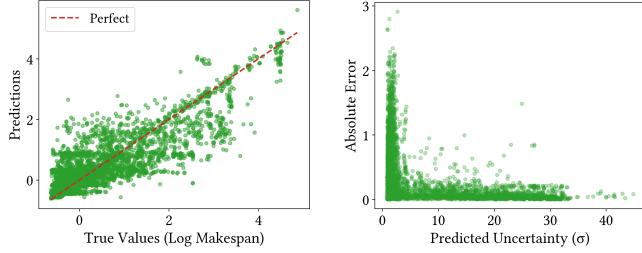
J.1 Best Configuration

The optimal configuration (hp_036) uses:

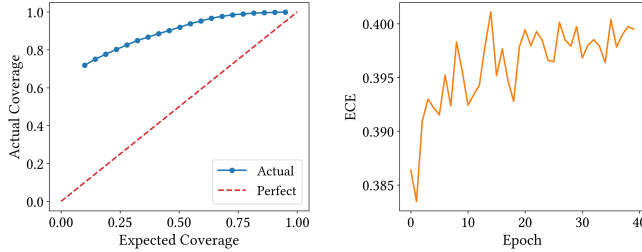
- $\lambda_{MSE} = 20.0$ (MSE loss weight)
- $\lambda_{NS} = 0.001$ (Non-saturating regularization)
- Learning rate = 0.0005
- Hidden dimension = 128
- GAT layers = 3, Attention heads = 4



(a) Training and validation loss curves over epochs. (b) R^2 score progression during training.



(c) Predicted vs. true values ($R^2 = 0.81$). (d) Uncertainty vs. prediction error.



(e) Calibration curve (ECE=0.396). (f) ECE progression during training.

Figure 2: NIG training results: (a-b) loss and R^2 curves showing convergence, (c) prediction accuracy with $R^2 = 0.81$, (d-f) uncertainty calibration metrics.

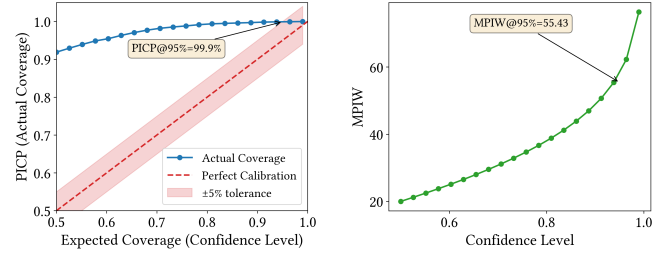
J.2 Detailed Metrics

J.3 Interpretation

The NIG model achieves **conservative calibration**: at 95% confidence, the prediction intervals contain 99.9% of true values (4.9% over-coverage). This is intentional and desirable for safety-critical embedded scheduling; the model never produces overconfident predictions that could lead to thermal violations.

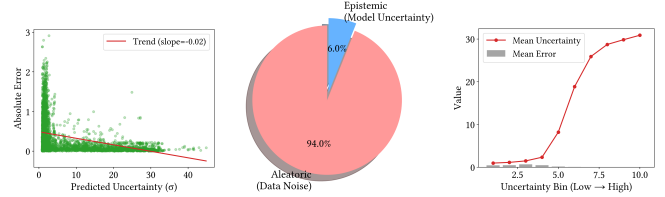
The Spearman $\rho = 0.946$ indicates strong **ranking capability**, which is essential for scheduling: the model correctly orders configurations by expected makespan, enabling effective scheduling decisions even when point predictions have residual error.

The 94%/6% **aleatoric/epistemic split** shows the model correctly attributes most uncertainty to irreducible data noise (OS scheduling jitter, thermal variation) rather than model uncertainty, indicating the architecture is appropriately complex for the task.



(a) PICP vs. expected coverage. (b) MPIW vs. confidence level.

Figure 3: Prediction Interval Coverage Probability (PICP) analysis: (a) conservative calibration where actual coverage exceeds nominal at all confidence levels, (b) Mean Prediction Interval Width increases appropriately with confidence.



(a) Uncertainty-error (b) Aleatoric (94%) vs. epistemic (6%). (c) Binned calibration.

Figure 4: Uncertainty decomposition: (a) positive correlation between uncertainty and prediction error validates uncertainty estimates, (b) 94%/6% aleatoric/epistemic split shows irreducible noise dominates, (c) higher uncertainty bins exhibit higher errors confirming calibration.

J.4 Hyperparameter Sensitivity Analysis

Table 18 reports sensitivity to key architectural hyperparameters. We vary one parameter while holding others at their optimal values.

Key findings: (1) $L = 3$ layers provides the best balance—fewer layers underfit, while more layers cause over-smoothing on our relatively small graphs. (2) $H = 4$ attention heads capture diverse interaction patterns without overfitting. (3) Non-saturating regularization λ_{NS} critically affects calibration: without it, PICP drops to 87.3% (under-coverage), demonstrating the importance of Wu et al.’s correction for evidential regression. The default $\lambda_{NS} = 0.001$ achieves near-perfect calibration without sacrificing point accuracy.

K Discussion and Future Work

Key Insights. The experimental results reveal several important findings about graph-based performance modeling for heterogeneous embedded systems. First, structural information significantly improves ranking capability: the 12 percentage point improvement in Spearman ρ from MLP (0.83) to GraphPerf-RT (0.95) demonstrates that explicitly modeling task dependencies, resource topology, and their interactions captures performance-critical relationships that tabular features miss. Second, heterogeneous message passing outperforms homogeneous alternatives: the distinction between

Metric	Value
R^2 (coefficient of determination)	0.8055
RMSE (log scale)	0.449
MAE (log scale)	0.244
Spearman ρ (ranking correlation)	0.946
PICP @ 95% confidence	99.9%
Mean Prediction Interval Width	58.1
ECE (Expected Calibration Error)	0.396
MCE (Maximum Calibration Error)	0.618
Aleatoric Uncertainty (mean)	356.3
Epistemic Uncertainty (mean)	22.9
Epistemic Ratio	6.0%
Epochs (early stopped)	40/50
Best Validation R^2	0.809

Table 17: NIG Evidential Model: Full Test Set Metrics

Configuration	R^2	Spearman	PICP (%)
<i>GAT Layers</i>			
$L = 2$	0.77	0.91	98.2
$L = 3$ (default)	0.81	0.95	99.9
$L = 4$	0.80	0.94	99.7
$L = 6$	0.78	0.93	99.5
<i>Attention Heads</i>			
$H = 2$	0.78	0.92	99.1
$H = 4$ (default)	0.81	0.95	99.9
$H = 8$	0.80	0.94	99.6
<i>Non-Saturating Regularization λ_{NS}</i>			
$\lambda_{NS} = 0$ (no reg.)	0.81	0.95	87.3
$\lambda_{NS} = 0.001$ (default)	0.81	0.95	99.9
$\lambda_{NS} = 0.01$	0.80	0.94	99.8
$\lambda_{NS} = 0.1$	0.78	0.93	99.9

Table 18: Hyperparameter sensitivity analysis on validation set.

task-task, task-resource, and resource-resource edges enables the model to learn different interaction patterns for each relationship type. Third, CFG-derived code semantics complement runtime features: static analysis provides information about control flow complexity and memory access patterns that runtime counters alone cannot capture, improving generalization to unseen inputs. Fourth, conservative uncertainty calibration enables practical risk management: the 99.9% PICP at 95% confidence ensures the scheduler can reliably filter out potentially harmful configurations without excessive conservatism.

Limitations. Several limitations suggest directions for future work. Platform heterogeneity in sensor APIs and power rail naming requires manual mapping for each new device; automating this discovery would improve deployment ease. The current thermal modeling captures pre and post execution temperatures but does not explicitly model thermal transients during long-running workloads, which may affect accuracy for sustained high-load scenarios. The framework currently targets OpenMP task parallelism; extending to other programming models such as CUDA for GPU workloads or OpenCL for portable heterogeneous computing would

broaden applicability. Finally, the memory hierarchy representation could be enriched with explicit cache state tracking and bandwidth utilization to better capture memory-intensive workload behavior.

Broader Impact. Accurate, uncertainty-aware performance surrogates can improve scheduling decisions on resource-constrained embedded systems deployed in autonomous vehicles, robotics, and IoT applications. By enabling risk-aware DVFS and core allocation, GraphPerf-RT helps reduce energy consumption and prevent thermal violations that could cause system instability or hardware damage in safety-critical deployments. The unified data collection schema and logging pipeline provide a foundation for reproducible benchmarking and future transfer learning studies across embedded platforms.

Future Directions. Beyond the platform extensions discussed in Appendix I.12, several research directions merit exploration: (1) online adaptation via continual learning to handle workload drift without full retraining; (2) integration with compiler-level optimizations to jointly optimize code generation and scheduling; (3) extension to multi-tenant scenarios with interference modeling between co-located applications; and (4) deployment on emerging heterogeneous architectures combining CPUs, GPUs, and domain-specific accelerators (NPUs, TPUs).

L Extended Experimental Setup Details

This section provides extended details on our experimental setup, including hardware platform configurations, benchmark suite descriptions, data collection infrastructure, and baseline evaluation protocols.

L.1 Hardware Platform Details

We evaluate GraphPerf-RT on three embedded ARM SoCs that represent diverse heterogeneous architectures and thermal management capabilities.

NVIDIA Jetson TX2 features a heterogeneous hexa-core configuration combining two high-performance Denver 2 cores (ARMv8, 64-bit, out-of-order) with four energy-efficient ARM Cortex-A57 cores. The platform supports 12 discrete DVFS levels ranging from 345.6 MHz to 2.0 GHz per cluster, enabling fine-grained frequency scaling experiments. Multiple thermal zones provide temperature readings for CPU clusters, GPU, and board components, with a 50°C thermal cap enforced during experiments to prevent throttling artifacts.

RUBIK Pi is an 8-core single-board computer based on Cortex-A72-class cores with per-core userspace DVFS capability. This platform represents lower-cost embedded computing scenarios with limited thermal headroom. Energy measurement uses the `qcom-battmgr` battery management interface or `hwmon` power rails, with power samples integrated at 0.5 second intervals to compute energy in Joules.

NVIDIA Jetson Orin NX features an octa-core ARM Cortex-A78AE CPU representing the latest generation of embedded AI computing platforms. The platform provides 8 homogeneous high-performance cores with userspace

DVFS control across multiple frequency levels. Compared to TX2, Orin NX offers higher single-thread performance and improved power efficiency, enabling evaluation of GraphPerf-RT on modern embedded hardware with different performance-power trade-offs.

System Configuration. Across all platforms, we configure the Linux `cpufreq` subsystem for userspace governor control. Available DVFS indices are discovered dynamically from the kernel interface (`scaling.available_frequencies`), and actual per-core frequencies are verified by reading `scaling_cur_freq` after each configuration change. Thermal zone temperatures are sampled from the `sysfs` thermal interface before and after each benchmark execution to capture thermal dynamics. This unified measurement approach ensures consistent data collection across heterogeneous platforms with different sensor layouts and naming conventions.

L.2 Benchmark Suite Details

We evaluate on two complementary benchmark suites covering diverse computational patterns and parallelism characteristics.

Barcelona OpenMP Tasks Suite (BOTS) provides 12 task-parallel applications designed to stress OpenMP runtime schedulers: *alignment* (dynamic programming for sequence alignment), *fft* (recursive fast Fourier transform), *fib* (recursive Fibonacci with fine-grained tasks), *floorplan* (branch-and-bound optimization), *health* (discrete event simulation), *concom* (connected components in graphs), *knapsack* (combinatorial optimization), *nqueens* (constraint satisfaction with backtracking), *sort* (parallel merge sort), *sparselu* (sparse LU factorization), *strassen* (matrix multiplication), and *uts* (unbalanced tree search with irregular parallelism). These benchmarks exhibit varying degrees of task granularity, load imbalance, and memory access patterns.

PolyBench contributes 30 additional kernels spanning linear algebra (*2mm*, *3mm*, *gemm*, *gemver*, *gesummv*, *symm*, *syrk*, *syr2k*, *trmm*, *cholesky*, *durbin*, *lu*, *ludcmp*, *trisolv*), stencil computations (*jacobi-1d*, *jacobi-2d*, *seidel-2d*, *fdtd-2d*, *heat-3d*), data mining (*correlation*, *covariance*), and medley applications (*atax*, *bicg*, *doitgen*, *mvt*, *floyd-warshall*, *nussinov*, *deriche*, *adi*, *gramschmidt*). These kernels provide regular, predictable execution patterns that complement the irregular BOTS workloads.

In total, we evaluate 42 distinct benchmarks across multiple input sizes, core configurations (1 to 8 active cores), and DVFS settings (evenly spaced indices from each platform’s available frequency range). Both sequential and task-parallel execution modes are profiled, resulting in 73,920 samples.

L.3 Data Collection Infrastructure

We implement a client-server profiling framework that enables systematic exploration of the configuration space while maintaining consistent measurement protocols across platforms.

The **device client** executes on each embedded board and manages the complete profiling sequence for each run. It first configures the `cpufreq` userspace governor and enables the

specified core subset via `cpuset` control. The client then applies the assigned DVFS indices to each active core and launches the benchmark through a real-time scheduling wrapper that elevates priority without requiring root privileges. During execution, it collects hardware performance counters using the Linux `perf_event` interface, including cycles, instructions, cache references, cache misses, branches, branch misses, task clock, and CPU clock. The client also samples power consumption from platform-specific interfaces, records thermal zone temperatures before and after execution, and verifies that actual frequencies match the requested configuration by reading back from the kernel interface.

The **host server** orchestrates the experimental campaign by generating action tuples $a = (m, f, p)$ representing core mask, DVFS vector, and optional priority settings. Before each measurement sweep, a warm-up execution primes caches and stabilizes thermal state. Each completed run produces one CSV row containing timestamp, iteration index, execution mode, assigned and measured frequencies, active core configuration, input parameters, wall-clock execution time, per-rail energy and power readings, all performance counter values, and thermal zone temperatures.

L.4 Graph Extraction Pipeline

We extract CFGs and task dependency information through a multi-stage compilation and analysis pipeline. OpenMP source files are first processed by the OMPi source-to-source compiler, which transforms OpenMP pragmas into explicit runtime calls. The ALF-llvm backend then generates both LLVM intermediate representation (`*.ll`) and ARTIST2 Language for Flow analysis files (`*.alf`). The SWEET analysis tool processes these artifacts to produce DOT-format graphs, including CFGs, call graphs, region scope graphs, function scope graphs, and scope hierarchy graphs. A post-processing stage maps ALF entities to OpenMP tasks, merges chains of trivially small tasks while preserving dependency provenance, and computes topological encodings including depth, distance-to-sink, and centrality measures.

Data Preprocessing. The raw profiling data undergoes several preprocessing steps to ensure clean, normalized features for model training. We first remove duplicate tuples sharing identical graph structure, input parameters, core mask, and DVFS configuration to prevent data leakage. Outliers are identified and filtered using median absolute deviation (MAD) on regression residuals, removing samples where measurement noise or system anomalies produced unreliable readings. Performance counter windows are aligned to execution phases to ensure temporal consistency across samples.

Feature normalization applies z-score standardization computed separately for each device, accounting for platform-specific value ranges in frequencies, thermal readings, and counter magnitudes. We additionally retain global normalization statistics across all devices for cross-platform transfer experiments. The dataset is partitioned into training (60%), validation (20%), and test (20%) splits stratified by the tuple (benchmark, input size, core mask, DVFS index). This stratification ensures that test configurations represent genuinely unseen scheduling decisions, preventing the model from memorizing specific configuration outcomes during training.

L.5 Baseline Models and Metrics

Baseline Models. We compare GraphPerf-RT against five baseline approaches spanning traditional machine learning, standard neural networks, and graph-based architectures. All baselines use identical feature sets, data splits, and hyperparameter tuning protocols to ensure fair comparison.

Linear Regression serves as a simple baseline using Ridge regularization on flattened tabular features extracted from task graphs and system state.

Random Forest provides an ensemble baseline with 100 decision trees, where tree depth and minimum samples per leaf are tuned on the validation set.

Multi-Layer Perceptron (MLP) uses a three-layer feed-forward architecture operating on the same tabular features as Linear Regression and Random Forest, without any graph structure information.

Graph Convolutional Network (GCN) applies homogeneous message passing where all nodes and edges are treated uniformly, losing the type distinctions between task and resource nodes.

Heterogeneous Graph Transformer (HGT) [Hu *et al.*, 2020] represents a state-of-the-art heterogeneous GNN that uses type-aware attention over our node and edge schema. This baseline isolates the contribution of our evidential heads and runtime context integration, as HGT lacks evidential uncertainty quantification, thermal and utilization context in node features, and our CFG-derived task features.

Prediction Metrics. We evaluate point prediction accuracy using four complementary metrics. Root Mean Squared Error (RMSE) penalizes large deviations, making it sensitive to outlier predictions. Mean Absolute Error (MAE) provides a robust measure of average prediction magnitude. Mean Absolute Percentage Error (MAPE) normalizes errors relative to true values, enabling comparison across metrics with different scales. Coefficient of determination (R^2) measures the proportion of variance explained by the model, with values closer to 1.0 indicating better fit.

Ranking Metrics. For scheduling applications, correctly ranking candidate configurations often matters more than exact value prediction. Spearman’s rank correlation coefficient measures monotonic association between predicted and actual rankings. Kendall’s τ counts concordant versus discordant pairs, providing a robust ranking measure. Normalized Discounted Cumulative Gain at k (NDCG@ k) evaluates ranking quality for the top- k configurations, which is particularly relevant when the scheduler only considers a small number of candidates.

Uncertainty Metrics. Calibration quality determines whether predicted confidence intervals are trustworthy for risk-aware scheduling. Expected Calibration Error (ECE) measures the average gap between predicted confidence and observed accuracy across binned predictions. Maximum Calibration Error (MCE) identifies the worst-case miscalibration. Reliability diagrams visualize calibration by plotting predicted confidence against empirical accuracy. Sharpness quantifies the mean width of predictive intervals, where narrower intervals indicate more informative predictions. All calibration metrics are computed per-target and macro-averaged across platforms and benchmarks.

L.6 Extended Results Analysis

Overall Prediction and Ranking Performance. The progression from tabular baselines (Linear Regression, Random Forest, MLP) to graph-based methods (GCN, HGT, GraphPerf-RT) reveals the importance of structural information. The MLP baseline achieves Spearman=0.83; GCN improves to 0.87; HGT reaches 0.89; GraphPerf-RT achieves 0.95 by combining heterogeneous message passing with evidential uncertainty heads and runtime context integration.

Platform-Specific Analysis. Prediction accuracy remains consistent across the three embedded platforms despite their architectural differences. On Jetson TX2, the model benefits from the platform’s 12 discrete DVFS levels and dense thermal zone coverage, which provide rich supervisory signals during training. Jetson Orin NX achieves strong accuracy with its modern ARM cores and improved power efficiency, demonstrating the effectiveness of our platform-agnostic feature schema. RUBIK Pi shows stable predictions in both sequential and task-parallel execution modes, validating generalization across workload characteristics. Per-device z-score normalization combined with device sheet broadcasting contributes to this cross-platform stability.

Uncertainty Calibration Details. The evidential regression heads provide conservatively calibrated uncertainty estimates. At 95% confidence, the Prediction Interval Coverage Probability (PICP) reaches 99.9%, meaning true values fall within predicted intervals more often than the nominal rate. This conservative calibration is intentional and desirable for safety-critical embedded scheduling: the model produces slightly wider intervals than strictly necessary, ensuring decisions never rely on overconfident predictions. The uncertainty decomposition shows 94% aleatoric (data-driven) and 6% epistemic (model) uncertainty, indicating the model correctly identifies irreducible noise from OS scheduling and hardware variability as the dominant uncertainty source.

Ablation Study Details. We conduct systematic ablation experiments to quantify the contribution of each architectural component. Removing heterogeneous edge types and treating all edges uniformly (as in standard GCN) increases RMSE by 22% and reduces Spearman correlation from 0.95 to 0.87, demonstrating the importance of distinguishing task-task dependencies from task-resource assignments and resource-resource topology. Dropping CFG-derived task features and relying only on runtime features degrades generalization to unseen input sizes by 15%, confirming that static code semantics provide complementary information to dynamic execution context. Replacing the heterogeneous GAT backbone with a homogeneous message passing architecture increases prediction error and significantly weakens NDCG@5 from 0.94 to 0.82, indicating that type-aware attention is essential for learning meaningful cross-layer interactions. Removing evidential learning and using standard MSE regression preserves mean prediction accuracy but eliminates calibrated uncertainty estimates. Without uncertainty gating, the proportion of unsafe scheduling proposals (those that would cause thermal violations) increases from 3% to 18%, demonstrating the practical value of calibrated confidence intervals for risk-aware decision making.

Cross-Platform Generalization. To evaluate transfer ca-

pabilities, we train GraphPerf-RT on data from two platforms and evaluate on the held-out third platform without fine-tuning (see Table 6). This leave-one-platform-out protocol tests whether learned representations generalize beyond specific architectures. Results show significant degradation for raw makespan prediction (R^2 drops to 0.28–0.56, or negative for TX2), consistent with the challenges of heterogeneous ARM platforms with different core counts and thermal characteristics. However, ranking quality degrades less severely than point prediction: relative ordering of configurations remains useful for scheduling guidance even when absolute predictions are inaccurate. Device sheet broadcasting and per-device feature normalization partially mitigate domain shift. These results highlight cross-platform transfer as a known limitation; practical deployment requires platform-specific fine-tuning or adaptation.

L.7 Inference Latency Details

Table 19 reports end-to-end inference latency across all three embedded platforms.

Platform	Latency (ms)	Memory (MB)	Throughput
Jetson TX2 (GPU)	2.1 ± 0.3	12.4	476 samples/s
Jetson Orin NX (GPU)	4.3 ± 0.5	12.4	233 samples/s
RUBIK Pi (CPU)	6.4 ± 0.8	12.4	156 samples/s

Table 19: End-to-end inference latency across embedded platforms (batch=1, typical 8-node graph). Includes graph construction and feature extraction.

On-device inference completes in 2–7ms for typical task graphs (8 nodes, 56 edges), enabling real-time configuration evaluation during scheduling decisions. Model size is 12.4MB (TorchScript checkpoint), fitting within embedded memory constraints. The GAT architecture achieves sub-linear scaling with graph size due to sparse attention patterns, with computational cost per layer $O(H \cdot |E| \cdot d)$ where H is attention heads, $|E|$ is edge count, and d is hidden dimension.

L.8 World Model Design Rationale

We use GraphPerf-RT as a *deterministic transition model with epistemic uncertainty quantification*. Unlike stochastic world models that learn full transition distributions, our formulation predicts point estimates $\hat{y} = \gamma$ with calibrated uncertainty bounds from the NIG posterior. This design choice is deliberate: embedded scheduling requires fast inference (2–7ms), and the evidential framework provides uncertainty awareness without the sampling overhead of probabilistic models. The epistemic uncertainty signals out-of-distribution inputs, enabling conservative action gating during synthetic rollouts.

L.9 D3QN Algorithm Selection Rationale

We select Dueling Double DQN (D3QN) over policy gradient methods (PPO, SAC) for three reasons:

1. **Sample efficiency:** D3QN’s off-policy nature with experience replay integrates naturally with Dyna-style planning where synthetic rollouts augment the replay

buffer, whereas on-policy PPO discards data after each update.

2. **Discrete action space:** DVFS scheduling involves selecting from discrete frequency levels and core masks, and D3QN handles this naturally while PPO/SAC require discretization or continuous relaxation.
3. **Stability on embedded platforms:** D3QN’s deterministic policy with ϵ -greedy exploration produces more consistent behavior than stochastic policies, reducing variance in thermal-sensitive environments.

Empirically, D3QN with GraphPerf-RT converges $2.5\times$ faster than PPO baselines in our preliminary experiments.

L.10 Extended RL Evaluation

RL Methods. (1) **SAMFRL:** Single-Agent Model-Free RL using standard Q-learning without a learned world model. (2) **SAMBRL:** Single-Agent Model-Based RL using GraphPerf-RT as the environment model for synthetic rollouts. (3) **MAMFRL-D3QN:** Multi-Agent Model-Free RL with Dueling Double DQN, where each core is an agent. (4) **MAMBRL-D3QN:** Multi-Agent Model-Based RL with Dueling Double DQN, using GraphPerf-RT for Dyna-style planning.

Experimental Protocol. Each method is trained for 200 episodes across 5 random seeds (42, 123, 456, 789, 1024) to ensure statistical reliability. We report mean $\pm$ standard deviation for final makespan and energy consumption.

State Space. The observation at each timestep consists of: (1) current per-core DVFS indices (normalized to $[0, 1]$), (2) per-core utilization (exponential moving average), (3) per-core thermal readings (normalized by $T_{\max}$), (4) graph-level summary from GraphPerf-RT (pooled task node embeddings capturing workload characteristics), and (5) remaining task count. For multi-agent methods, each agent (core) receives local observations plus a global context vector.

Action Space. The action space consists of per-core DVFS index selection (discrete, 12 levels on TX2) and core mask configuration (binary, which cores are active). For single-agent methods, actions are joint across all cores. For multi-agent methods (MAMFRL-D3QN, MAMBRL-D3QN), each agent selects its own DVFS index with coordination through shared value decomposition.

Reward Function. The reward signal balances performance and safety:

$$r_t = -w_{\text{time}} \cdot \hat{t}_{\text{makespan}} - w_{\text{energy}} \cdot \hat{e} - w_{\text{thermal}} \cdot \max(0, T_t - T_{\text{soft}})^2$$

where $\hat{t}_{\text{makespan}}$ and $\hat{e}$ are normalized makespan and energy predictions from GraphPerf-RT (for model-based) or actual measurements (for model-free), T_t is current temperature, $T_{\text{soft}} = 45^\circ\text{C}$ is the soft thermal threshold, and weights $(w_{\text{time}}, w_{\text{energy}}, w_{\text{thermal}}) = (1.0, 0.5, 10.0)$ prioritize makespan while heavily penalizing thermal violations.

Detailed Results. In the single-agent setting, SAMFRL (model-free) slightly outperforms SAMBRL (model-based), suggesting that model accuracy may limit single-agent planning. However, in the multi-agent setting, the model-based

approach (MAMBRL-D3QN) significantly benefits from co-ordinated planning with GraphPerf-RT as the shared world model.

Convergence Analysis. Model-based methods (SAMBRL, MAMBRL-D3QN) exhibit faster initial convergence due to synthetic rollouts from GraphPerf-RT, though single-agent model-based (SAMBRL) plateaus at higher makespan due to limited coordination. Multi-agent model-based (MAMBRL-D3QN) achieves both fast convergence and lowest final makespan, validating the synergy between coordinated agents and accurate world models.

Computational Complexity. Model-free methods (SAMBRL, MAMBRL-D3QN) require $O(N)$ real samples for learning. Model-based methods (SAMBRL, MAMBRL-D3QN) add $O(S)$ synthetic samples from the world model, increasing total training time but reducing on-device exploration. GraphPerf-RT itself provides zero-shot prediction with $O(1)$ inference complexity, enabling direct use without additional learning when deployed as a standalone evaluator.

Runtime Integration Protocol. At runtime, we enumerate feasible $\mathcal{A}$ under availability and thermal caps, score candidates with GraphPerf-RT, and gate by epistemic uncertainty and predicted intervals:

$$\text{gate}(a) = (\text{Epi}_{\text{time}}(a) \leq \eta) \wedge (\text{PI}_{\text{time}}^{(1-\delta)}(a) \leq T_{\text{max}}).$$

We rank by $\hat{y}_{\text{time}}$ and execute the top action. Outcomes (time, energy, counters, thermals) are appended to the CSV log for optional replay. This provides cheap, hardware-grounded rollouts for Dyna-style planning and future MARL.

M Extended Design Details

This section provides extended details on the GraphPerf-RT architecture, including the intuition behind our design choices, detailed node and edge type specifications, and the complete training procedure.

M.1 Design Intuition

Why Task-Resource-Memory Decomposition? Performance of parallel applications on heterogeneous SoCs arises from three interacting factors: (1) *what computation runs* (task structure, dependencies, code complexity), (2) *where it runs* (core types, frequencies, thermal state), and (3) *how data flows* (cache hierarchies, memory bandwidth, contention). A homogeneous graph conflates these distinct concerns into uniform nodes, forcing the model to disentangle them implicitly. Our heterogeneous decomposition makes these factors explicit: **Task nodes** capture application semantics (CFG complexity, DAG structure, data dependencies), **Resource nodes** capture hardware state (DVFS settings, utilization, thermals), and **Memory nodes** capture the memory hierarchy (cache levels, bandwidth constraints). The typed edges then encode the *interactions* that determine performance: task-task edges model parallelism and synchronization overhead, task-resource edges model scheduling decisions and affinity, resource-resource edges model shared cache contention between cores, and resource-memory edges model bandwidth bottlenecks. This explicit structure allows the GNN

to learn interpretable attention patterns, for example, attending strongly to thermal headroom when predicting energy, or to critical-path edges when predicting makespan, rather than discovering these relationships from scratch in a homogeneous representation.

M.2 Node Type Specifications

Task Nodes (V_T): Each OpenMP task is represented as a task node in the graph. Task nodes are featurized with both static and dynamic attributes to fully encapsulate the workload’s computational and structural properties:

- **CFG Features:** We extract the CFG from the source code associated with each task using ALF-llvm and SWEET, then compute hand-crafted features via AST parsing. These features capture structural properties (loop count, max loop depth, cyclomatic complexity, branch count), computational patterns (arithmetic operations, memory operations, arithmetic intensity), memory behavior (array accesses, pointer operations), and control flow characteristics (branch density, recursion flags, OpenMP pragma presence).
- **Static Features:** Task-level characteristics including estimated instruction count (from compiler IR), memory footprint (bytes allocated or accessed), parallelization degree (e.g., number of subtasks spawned), branch proxies (e.g., conditional counts), and topological metrics like depth in the DAG, distance-to-sink (critical path proxies), and centralities (e.g., betweenness to indicate bottleneck potential).
- **Dynamic Features:** Runtime-dependent attributes such as input data size (affecting memory intensity), iteration counts (for loops with variable bounds), dependency fan-in/fan-out (indicating parallelism width), recent performance counter snapshots (e.g., cycles per instruction from prior runs), run-mode flags (e.g., sequential vs. parallel), and thermal footprint on hosting cores (e.g., estimated heat generation based on operation types).

Resource Nodes (V_R): Each processing core in the heterogeneous system is represented as a resource node, encoding both architectural characteristics and dynamic state information to model hardware heterogeneity and runtime variability:

- **Architectural Features:** Core type (e.g., big vs. LITTLE in big.LITTLE architectures), cache hierarchy specifications (e.g., L1/L2 sizes, associativity), supported instruction sets (e.g., NEON/SVE flags), peak computational capacity (e.g., FLOPS at max frequency), cluster ID, and interconnect proxies (e.g., bandwidth to shared resources).
- **Dynamic Features:** Current frequency setting (DVFS index or one-hot encoded), utilization level (EMA over recent intervals), temperature (from thermal zones), thermal headroom (computed as $T_{\text{max}} - T_{\text{current}}$, where T_{max} is the throttling threshold), trend (e.g., ΔT over last samples), and bandwidth proxy (e.g., estimated memory throughput under current load).
- **Power Characteristics:** Power consumption models (e.g., quadratic approximations of power vs. frequency)

and DVFS efficiency curves (e.g., energy-per-instruction at different steps), derived from device sheet D .

Memory Nodes (V_M): Memory hierarchy components (L1/L2/L3 caches, main memory) are represented as memory nodes to capture memory subsystem characteristics and contention effects, including level (e.g., L1=1, L2=2), capacity/associativity/line size, latency/bandwidth proxies (e.g., cycles per access, GB/s), and contention indicators (e.g., shared vs. private).

M.3 Edge Type Specifications

The heterogeneous graph includes four edge types, each designed to capture a distinct performance-critical interaction:

Task-Task Edges (E_{TT}): Directed edges representing precedence constraints between tasks, derived from the original OpenMP dependency graph, with attributes such as type (spawn/join/data), critical-edge flag (1 if on longest path), hop distance, and contention proxies (e.g., data volume transferred). *Rationale:* The critical path through E_{TT} edges determines the theoretical minimum makespan; fork/join patterns reveal synchronization overhead; data dependency volumes indicate communication costs.

Task-Resource Edges (E_{TR}): Bidirectional edges connecting tasks to the resources on which they execute or could potentially execute, encoding the current scheduling assignment under mask m and DVFS f , with attributes like affinity strength (e.g., based on core type suitability) and placement cost (e.g., migration overhead proxy). *Rationale:* Task-to-core affinity affects cache locality (big cores vs. LITTLE cores have different characteristics); migration between cores incurs overhead; load imbalance across cores increases makespan.

Resource-Resource Edges (E_{RR}): Undirected edges between resource nodes that share hardware components (e.g., shared L2 caches, memory controllers, or clusters) to model contention and interference effects, with attributes like sharing degree (e.g., number of shared ways) and interconnect latency. *Rationale:* Cores sharing an L2 cache can benefit from data reuse but also contend for cache capacity; cores in the same cluster share DVFS domains; inter-cluster communication has higher latency.

Resource-Memory Edges (E_{RM}): Directed edges connecting processing cores to memory hierarchy components, enabling the model to reason about memory access patterns and bandwidth constraints, with attributes such as access frequency (e.g., expected loads/stores) and bandwidth allocation. *Rationale:* Memory-bound workloads are bottlenecked by bandwidth, not compute; cache miss rates determine effective memory latency; understanding which level services most accesses predicts energy and time.

This rich representation ensures the model captures cross-layer interactions and enables the GNN to learn distinct attention patterns for each interaction type.

M.4 GNN Architecture Details

GraphPerf-RT employs a GAT architecture specifically designed to handle heterogeneous graphs with multiple node and edge types. The architecture consists of several key components, with 3–6 layers, using hidden dimensions typically

in [128, 256] for embeddings, and multi-head attention (e.g., 4–8 heads) to stabilize learning.

Type-Specific Encoders:

$$\mathbf{h}_v^{(0)} = \text{MLP}_{\text{type}(v)}(\mathbf{x}_v; \theta_{\text{type}(v)}) \quad (10)$$

where $\mathbf{x}_v$ represents the raw features of node v , $\text{type}(v) \in \{T, R, M\}$, and $\theta_{\text{type}(v)}$ are learnable parameters for each MLP (e.g., 2–3 layers with ReLU activations and dropout $p = 0.1$). Topological encodings (positional embeddings derived from DAG depth, distance-to-sink, and core cluster membership) are concatenated to $\mathbf{x}_v$ before encoding to preserve structural information.

Heterogeneous GAT Layers:

$$\mathbf{h}_v^{(l+1)} = \sigma \left(\mathbf{h}_v^{(l)} + \sum_{r \in \mathcal{R}} \sum_{u \in \mathcal{N}_r(v)} \alpha_{uv,r}^{(l)} \mathbf{W}_r^{(l)} \mathbf{h}_u^{(l)} \right) \quad (11)$$

where $\mathcal{R} = \{TT, TR, RR, RM\}$ is the set of edge types, $\mathcal{N}_r(v)$ is the neighborhood of v under edge type r , $\alpha_{uv,r}^{(l)}$ is the attention coefficient, $\mathbf{W}_r^{(l)}$ is a type-specific linear transformation matrix, and σ is a non-linearity (e.g., ELU). Residual connections are added for deeper models to mitigate vanishing gradients.

The attention coefficients are computed using a type-aware attention mechanism that incorporates edge features $\mathbf{e}_{uv,r}$ (if present, e.g., contention proxies):

$$e_{uv,r}^{(l)} = \text{LeakyReLU} \left(\mathbf{a}_r^\top [\mathbf{W}_r^{(l)} \mathbf{h}_u^{(l)} \parallel \mathbf{W}_r^{(l)} \mathbf{h}_v^{(l)} \parallel \phi_r(\mathbf{e}_{uv,r})] \right) \quad (12)$$

$$\alpha_{uv,r}^{(l)} = \frac{\exp(e_{uv,r}^{(l)})}{\sum_{r' \in \mathcal{R}} \sum_{w \in \mathcal{N}_{r'}(v)} \exp(e_{vw,r'}^{(l)})} \quad (13)$$

where $\mathbf{a}_r$ is a type-specific attention vector, $\parallel$ denotes concatenation, and ϕ_r is an optional edge feature projector (e.g., a 1-layer MLP with ReLU). Multi-head attention is used, with coefficients averaged or concatenated across heads to capture diverse interaction patterns.

Graph-Level Pooling:

$$\mathbf{h}_G = \text{CONCAT}(\text{POOL}_T(\{\mathbf{h}_v^{(L)} : v \in V_T\}), \text{POOL}_R(\{\mathbf{h}_v^{(L)} : v \in V_R\}), \text{POOL}_M(\{\mathbf{h}_v^{(L)} : v \in V_M\})) \quad (14)$$

where $\text{POOL}_{\text{type}}$ are learnable (e.g., via attention) or fixed aggregators (e.g., mean), yielding a fixed-size $\mathbf{h}_G$ (e.g., dimension 256–512) regardless of graph size. This ensures scalability and enables batched inference.

M.5 Evidential Learning Details

To provide uncertainty-aware predictions, GraphPerf-RT employs an evidential learning framework. Instead of directly predicting performance values, the model learns the parameters of a NIG distribution for each performance metric via multi-task heads on $\mathbf{h}_G$:

$$(\gamma_k, \nu_k, \alpha_k, \beta_k) = \text{MLP}_k(\mathbf{h}_G; \theta_k) \quad (15)$$

for each metric $k \in \{\text{makespan, energy, cache misses, branch misses, utilization}\}$, where MLP_k is a shared trunk (e.g., 2 layers, 128 units) with task-specific outputs (e.g., linear layers with softplus activations on $\nu_k, \alpha_k > 1, \beta_k > 0$ to ensure valid distributions).

The mean prediction and uncertainty estimates are derived as:

$$\hat{y}_k = \gamma_k \quad (16)$$

$$\text{Aleatoric Uncertainty}_k = \frac{\beta_k}{\alpha_k - 1} \quad (17)$$

$$\text{Epistemic Uncertainty}_k = \frac{\beta_k}{\nu_k(\alpha_k - 1)} \quad (18)$$

$$\text{Total Variance}_k = \text{Aleatoric}_k + \text{Epistemic}_k \quad (19)$$

The model is trained by minimizing the negative log marginal likelihood of the evidential distribution, augmented with non-saturating uncertainty regularization [Wu *et al.*, 2024] (to prevent evidence contraction on high-error samples) and a ranking loss (e.g., pairwise on makespan to align with scheduling objectives):

$$\begin{aligned} \mathcal{L} = & \sum_{k=1}^K \sum_{i=1}^N \left[\frac{1}{2} \log \frac{\pi}{\nu_{k,i}} - \frac{\alpha_{k,i} - 1}{2} \log(2\beta_{k,i}(1 + \nu_{k,i}\epsilon_{k,i}^2)) \right. \\ & \left. + (\alpha_{k,i} + \frac{1}{2}) \log(1 + \nu_{k,i}\epsilon_{k,i}^2) + \log \frac{\Gamma(\alpha_{k,i})}{\Gamma(\alpha_{k,i} + 1/2)} \right] \\ & + \lambda_{NS} \sum_k \mathcal{L}_{NS}(y_{k,i}, \gamma_{k,i}, \nu_{k,i}, \sigma_{k,i}) + \rho \mathcal{L}_{\text{rank}} \quad (20) \end{aligned}$$

where $\epsilon_{k,i} = (y_{k,i} - \gamma_{k,i})^2 / (2\beta_{k,i})$, Γ is the gamma function, $\mathcal{L}_{NS} = \max(0, |y - \gamma| - k\sigma) \cdot \nu$ is the non-saturating regularizer that prevents overconfidence on high-error samples (with $k = 2$ for 95% coverage and $\sigma = \sqrt{\beta(\nu + 1)/(\nu(\alpha - 1))}$ the total predictive standard deviation), λ_{NS} (e.g., 0.01) weights the non-saturating term, and ρ (e.g., 0.1) weights the ranking loss.

Critically, the ranking loss operates on Lower Confidence Bounds $\text{LCB}_i = \gamma_i - k\sigma_i$ rather than point estimates γ_i : $\mathcal{L}_{\text{rank}} = \sum_{i,j:i \prec j} \max(0, \text{LCB}_i - \text{LCB}_j + \delta)$, where $i \prec j$ denotes that configuration i should be scheduled before j (lower makespan). This risk-aware formulation ensures the scheduler prioritizes configurations with both low predicted makespan and high confidence, avoiding overoptimistic selections that could violate thermal constraints.

M.6 Handling Dynamic and Irregular Workloads

Several benchmarks in our evaluation, notably *fib*, *nqueens*, and *uts*, exhibit recursive task creation, input-dependent DAG structures, and highly irregular execution patterns. Our approach addresses these challenges through two complementary mechanisms. First, **CFG features** capture the control-flow complexity of each task’s source code, including recursive call patterns, nested conditionals, and loop structures with variable bounds; these embeddings encode the *potential* for irregular behavior even when the static DAG appears simple. Second, the **evidential framework** naturally produces higher epistemic uncertainty for irregular workloads: when

the model encounters task graphs whose structure or features differ significantly from training data (e.g., deep recursion in *uts*, combinatorial branching in *nqueens*), the learned ν parameter decreases, signaling lower confidence. This behavior is intentional and useful; rather than producing overconfident predictions for unpredictable workloads, GraphPerf-RT flags them with high uncertainty, enabling downstream schedulers to apply conservative policies (e.g., fallback to safe DVFS settings) or request additional profiling before committing to aggressive optimizations.

M.7 Extended Training Procedure

GraphPerf-RT is trained using a multi-stage procedure designed to handle the challenges of multi-task learning with uncertainty quantification, building on the data preprocessing (e.g., z-scoring per device, outlier filtering via MAD on residuals, 60/20/20 splits stratified by benchmark-input-mask-DVFS to avoid leakage):

Regularization Details. We apply graph edge dropout ($p = 0.1\text{--}0.2$), small feature noise ($\sigma = 0.05$), multi-scale graph training, and mild device augmentation (e.g., $\pm 5\%$ DVFS tables). Scheduling-aware ranking losses further align predictions with downstream policy selection, yielding fast inference, cross-device transfer capability, and useful uncertainties for downstream schedulers and RL.

Ablation Summary. The heterogeneous edge types contribute the largest ranking improvement (Spearman +8%), confirming that modeling task-resource interactions explicitly outperforms homogeneous message passing. Runtime telemetry adds +6% ranking improvement by capturing dynamic system behavior invisible to static analysis. CFG features contribute +4% by encoding control-flow complexity that predicts irregular execution patterns. Evidential heads minimally affect mean accuracy but are essential for conservative uncertainty calibration (PICP=99.9% at 95% confidence), enabling safe scheduling decisions.

N Extended Preliminaries

This section provides extended formal definitions and notation for the problem formulation, heterogeneous graph abstraction, and data collection pipeline.

N.1 Problem Formulation Details

This work addresses the problem of predicting multiple performance metrics for OpenMP task-parallel applications executing on heterogeneous embedded SoCs under varying DVFS configurations. The prediction model must provide accurate point estimates along with calibrated uncertainty quantification to enable risk-aware scheduling decisions.

Each OpenMP application is represented as a task DAG $G = (V, E)$ recovered from compiler artifacts and runtime instrumentation. A task $v \in V$ represents a unit of parallel work characterized by static code features extracted from the CFG, including loop counts, memory access patterns, and branch statistics. Additionally, each task carries optional runtime summaries from prior executions, such as performance counter snapshots and thermal footprints. An edge $e = (u \rightarrow v) \in E$ encodes a precedence relation derived

from OpenMP dependencies, classified as spawn, join, or data dependency types.

The target platform consists of C heterogeneous cores indexed by $i \in \{1, \dots, C\}$. Each core operates at discrete frequency levels $f_i \in \mathcal{F}_i$ determined by the DVFS subsystem. The immutable hardware characteristics are captured in the device sheet D , which includes core types and counts, cache hierarchy specifications (sizes, associativity, line sizes), DVFS tables per core cluster, governor support flags, thermal sensor layout and resolution, and toolchain version hashes for reproducibility. The device sheet remains constant throughout an episode and enables transfer learning across hardware generations.

The runtime state $\mathcal{S}$ captures dynamic system context that changes during execution. This includes the active core mask indicating which cores are available for scheduling, per-core DVFS indices and measured frequencies, utilization computed as an exponential moving average (EMA), thermal zone temperatures before and after execution, and recent performance counter values. These runtime features enable the model to adapt predictions based on current system conditions rather than relying solely on static analysis.

A scheduling action $a = (m, f, p)$ specifies the configuration under which a workload executes. The core mask $m \in \{0, 1\}^C$ selects active cores, the DVFS vector f assigns frequency levels with $f_i \in \mathcal{F}_i$ for each core, and the optional priority $p \in \{1, \dots, 99\}$ sets FIFO scheduling priority. All actions must satisfy feasibility constraints including core availability, affinity restrictions, and thermal caps that prevent execution when temperatures exceed safe thresholds.

Given a tuple $(G, D, \mathcal{S}, a)$, the surrogate model predicts a vector of performance metrics $y = (\text{makespan}, \text{energy}, \text{cache misses}, \text{branch misses}, \text{utilization})$. Lower values indicate better performance for makespan, energy, and miss metrics, while higher utilization is generally preferable. Each prediction carries calibrated uncertainty estimates decomposed into aleatoric uncertainty (irreducible noise from OS and co-runner interference) and epistemic uncertainty (model uncertainty that decreases with more training data).

Formally, we learn a mapping $\mathcal{M} : (G, D, \mathcal{S}, a) \mapsto (y, \mathcal{U})$, where $\mathcal{U}$ contains the NIG distribution parameters $(\gamma, \nu, \alpha, \beta)$ for each target metric. Point estimates are the predictive means $\hat{y}_k = \gamma_k$, while prediction intervals derive from the NIG posterior. The uncertainty decomposition follows standard evidential learning formulations with aleatoric uncertainty given by $\beta_k/(\alpha_k - 1)$ and epistemic uncertainty by $\beta_k/(\nu_k(\alpha_k - 1))$.

At scheduling time, candidate configurations (m, f) are scored in batch through the surrogate. Actions are ranked primarily by predicted makespan with uncertainty gates that filter out low-confidence proposals. Safe actions satisfying thermal constraints execute on the device, and outcomes (execution time, energy consumption, performance counters, thermal readings) are logged for continual model refinement.

N.2 Heterogeneous Graph Details

The key insight underlying our approach is that performance on heterogeneous embedded systems emerges from com-

plex interactions between application structure, hardware resources, and scheduling decisions. A unified heterogeneous graph representation enables explicit modeling of these cross-layer interactions through typed nodes and edges.

The heterogeneous graph contains three node types that capture distinct aspects of the system. Task nodes V_T represent OpenMP tasks and encode application semantics. Each task node carries CFG-derived features capturing control flow complexity, static code statistics including loop counts, bytes moved, and branch proxies, DAG topology metrics such as depth, distance to sink, and centrality measures, recent performance snapshots from prior runs, run mode flags distinguishing serial from parallel execution, and thermal footprint on hosting cores. Resource nodes V_R represent processing cores and encode hardware state. Each resource node captures the DVFS step as both index and one-hot encoding, core mask bit indicating active status, cluster ID for heterogeneous architectures, utilization computed as an exponential moving average, thermal headroom and temperature trend, and bandwidth proxy estimating memory throughput. Memory nodes V_M optionally represent cache hierarchy levels and encode cache level identifier, capacity, associativity and line size, and latency and bandwidth proxies.

Four edge types capture the performance-critical interactions between nodes. Task-task edges E_{TT} encode precedence constraints from the DAG with attributes including dependency type (spawn, join, or data), critical edge flag indicating membership on the longest path, hop distance, and contention proxies. Task-resource edges E_{TR} connect tasks to cores based on the scheduling assignment under the recorded mask and DVFS configuration, with attributes capturing affinity strength and migration overhead. Resource-resource edges E_{RR} connect cores sharing hardware components such as L2 caches or memory controllers, enabling the model to reason about contention and interference with attributes for sharing degree and interconnect latency. Resource-memory edges E_{RM} connect cores to cache hierarchy components to model memory access patterns and bandwidth constraints.

N.3 Data Pipeline Details

The data collection pipeline transforms OpenMP source code into heterogeneous graph representations paired with runtime performance measurements. The compilation stage processes OpenMP sources through the OMPi source-to-source compiler. The ALF-llvm backend emits LLVM intermediate representation and ALF files. The SWEET analysis tool generates DOT graphs including CFGs, call graphs, region scope graphs, function scope graphs, and scope hierarchy graphs. A post-processing stage maps ALF entities to OpenMP tasks, merges short task chains while preserving provenance, and attaches topological encodings computed via depth-first traversal.

Runtime logging captures one CSV row per execution with comprehensive telemetry. Each row records timestamp, iteration number, run mode (serial, tasks, or tied variants), assigned DVFS indices as frequency combination, measured per-core frequencies from the scaling subsystem, number of active cores, core string encoding the mask, input pa-

rameters, elapsed execution time, per-rail energy measurements and instantaneous power, performance counters including cycles, instructions, cache references, cache misses, branches, branch misses, task clock, CPU clock, and page faults, and thermal zone temperatures before and after execution. Derived features include temperature change ΔT , thermal headroom relative to throttling thresholds, and phase-aligned counter windows.

N.4 Learning Formulation Details

The model architecture consists of type-specific encoders followed by a heterogeneous GAT backbone and evidential prediction heads. Type-specific MLPs embed task, resource, and memory nodes into a common representation space, with separate parameter sets for each node type to handle their distinct feature schemas. A heterogeneous GAT with edge-type-specific transforms performs message passing over the four edge types E_{TT} , E_{TR} , E_{RR} , and E_{RM} . The number of attention layers ranges from 3 to 6 depending on graph complexity. Hierarchical pooling aggregates node embeddings per type and concatenates them to form the graph-level representation $\mathbf{h}_G$.

For each target metric k , an evidential head outputs NIG parameters $(\gamma_k, \nu_k, \alpha_k, \beta_k)$. The predictive mean is $\hat{y}_k = \gamma_k$. Uncertainty decomposes into aleatoric and epistemic components: $\text{Aleatoric}_k = \frac{\beta_k}{\alpha_k - 1}$ and $\text{Epistemic}_k = \frac{\beta_k}{\nu_k(\alpha_k - 1)}$. Training minimizes the negative log marginal likelihood of the NIG distribution with evidence regularization to prevent overconfidence and a ranking term aligned to makespan-first scheduling objectives.

References

- [Amini *et al.*, 2020] Alexander Amini, Wilko Schwarting, Ava Soleimany, and Daniela Rus. Deep evidential regression. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pages 14927–14937, 2020.
- [Blumofe and Leiserson, 1999] Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, 1999.
- [Brent, 1974] Richard P. Brent. The parallel evaluation of general arithmetic expressions. *Journal of the ACM*, 21(2):201–206, 1974.
- [Cormen *et al.*, 2009] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.
- [Gal and Ghahramani, 2016] Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48 of *Proceedings of Machine Learning Research*, pages 1050–1059, 2016.
- [Graham, 1969] R. L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969.
- [Hu *et al.*, 2020] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. In *Proceedings of The Web Conference 2020 (WWW '20)*, pages 2704–2710. ACM, 2020.
- [Kuleshov *et al.*, 2018] Volodymyr Kuleshov, Nathan Fenner, and Stefano Ermon. Accurate uncertainties for deep learning using calibrated regression. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 2796–2804, 2018.
- [Kwok and Ahmad, 1999] Yu-Kwong Kwok and Ishfaq Ahmad. Static scheduling algorithms for allocating directed task graphs to multiprocessors. *ACM Computing Surveys*, 31(4):406–471, 1999.
- [Lakshminarayanan *et al.*, 2017] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pages 6402–6413, 2017.
- [Morris *et al.*, 2019] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4602–4609, 2019.
- [Scarselli *et al.*, 2009] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2009.
- [Wu *et al.*, 2024] Yuefei Wu, Bin Shi, Bo Dong, Qinghua Zheng, and Hua Wei. The evidence contraction issue in deep evidential regression: Discussion and solution. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence*, volume 38, pages 21726–21734, 2024.
- [Xu *et al.*, 2019] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019.